
DSC 140A - Homework 02

Homework 02 Instructions. You may collaborate with whomever, including AI. Each student must submit their own homework even if they worked with others. We will not be grading for correctness, but we will be grading for completeness. Please turn out a single PDF file that contains your solutions to all of the problems. We recommend using L^AT_EX with our template. You can use markdown but make sure each problem is clearly labeled. For coding problem, we recommend using Jupyter notebooks, convert them into PDF, and then merge the PDF with your math problems solutions generated in L^AT_EX.

Problem 1.

The following is a common question you might see in an interview for a machine learning job. The question is: Should you scale your features before performing least squares?

Your friend thinks that scaling the features before performing least squares regression is a good idea. Their logic is that scaling sometimes improves the performance of k -nearest neighbors predictors, so it might help with least squares as well. But you're not so sure.

In this problem, we'll show that scaling the features before performing least squares regression does not actually change the predictions that are made, and so it doesn't improve the performance of the model¹.

Hint: It might be helpful to use some of the matrix-vector algebra properties covered in the math check. If you use a property that wasn't listed in the math check or lecture, it's OK, but make sure to explicitly state the property you're using.

- a) Let $(\vec{x}^{(1)}, y_1), (\vec{x}^{(2)}, y_2), \dots, (\vec{x}^{(n)}, y_n)$ be a training set of n feature vectors in $\mathbb{R}^d$ and their corresponding labels. Recall that the design matrix X is

$$X = \begin{pmatrix} 1 & \vec{x}_1^{(1)} & \vec{x}_2^{(1)} & \cdots & \vec{x}_d^{(1)} \\ 1 & \vec{x}_1^{(2)} & \vec{x}_2^{(2)} & \cdots & \vec{x}_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \vec{x}_1^{(n)} & \vec{x}_2^{(n)} & \cdots & \vec{x}_d^{(n)} \end{pmatrix}$$

Suppose we will scale feature i by a constant factor of c_i in each feature vector. The result is a scaled data set whose design matrix X_C is:

$$X_C = \begin{pmatrix} 1 & c_1 \vec{x}_1^{(1)} & c_2 \vec{x}_2^{(1)} & \cdots & c_d \vec{x}_d^{(1)} \\ 1 & c_1 \vec{x}_1^{(2)} & c_2 \vec{x}_2^{(2)} & \cdots & c_d \vec{x}_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & c_1 \vec{x}_1^{(n)} & c_2 \vec{x}_2^{(n)} & \cdots & c_d \vec{x}_d^{(n)} \end{pmatrix}$$

Find a diagonal matrix C so that $X_C = XC$.

Solution: We can find what the diagonal matrix C should be by writing out the matrix multiplication XC and comparing it to X_C .

First, how big should C be? Since X is $n \times (d+1)$, we need C to be $(d+1) \times (d+1)$ so that XC is also $n \times (d+1)$.

¹This is not to say that scaling is never useful when doing least squares regression. Scaling the features can make the weights easier to interpret, for example, or help with numerical stability.

Let's say that C is the diagonal matrix:

$$C = \begin{pmatrix} a_0 & 0 & 0 & \cdots & 0 \\ 0 & a_1 & 0 & \cdots & 0 \\ 0 & 0 & a_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a_d \end{pmatrix}$$

We're trying to find out what the a_i should be.

When we multiply XC , we get:

$$\begin{aligned} XC &= \begin{pmatrix} 1 & \vec{x}_1^{(1)} & \vec{x}_2^{(1)} & \cdots & \vec{x}_d^{(1)} \\ 1 & \vec{x}_1^{(2)} & \vec{x}_2^{(2)} & \cdots & \vec{x}_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \vec{x}_1^{(n)} & \vec{x}_2^{(n)} & \cdots & \vec{x}_d^{(n)} \end{pmatrix} \begin{pmatrix} a_0 & 0 & 0 & \cdots & 0 \\ 0 & a_1 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a_d \end{pmatrix} \\ &= \begin{pmatrix} a_0 & a_1 \vec{x}_1^{(1)} & a_2 \vec{x}_2^{(1)} & \cdots & a_d \vec{x}_d^{(1)} \\ a_0 & a_1 \vec{x}_1^{(2)} & a_2 \vec{x}_2^{(2)} & \cdots & a_d \vec{x}_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_0 & a_1 \vec{x}_1^{(n)} & a_2 \vec{x}_2^{(n)} & \cdots & a_d \vec{x}_d^{(n)} \end{pmatrix} \end{aligned}$$

Since we want this to equal X_C , we compare term by term and find that we should set:

$$\begin{aligned} a_0 &= 1 \\ a_1 &= c_1 \\ a_2 &= c_2 \\ &\vdots \\ a_d &= c_d \end{aligned}$$

Therefore, the matrix C is:

$$\begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & c_1 & 0 & \cdots & 0 \\ 0 & 0 & c_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & c_d \end{pmatrix}$$

- b) In lecture, we will see that for the unscaled data, the parameter vector that minimizes the mean squared error is given by $\vec{w}^* = (X^T X)^{-1} X^T \vec{y}$. When we scale the features, however, the optimal weight vector becomes $\vec{w}_C^* = (X_C^T X_C)^{-1} X_C^T \vec{y}$.

Show that $\vec{w}_C^* = C^{-1} \vec{w}^*$ by using the fact that $X_C = XC$.

Hint: $(AB)^{-1} = B^{-1}A^{-1}$ (as long as A and B are invertible).

Solution: We know that $X_C = XC$, so we can write:

$$\begin{aligned} \vec{w}_C^* &= (X_C^T X_C)^{-1} X_C^T \vec{y} \\ &= (CX^T XC)^{-1} CX^T \vec{y} \end{aligned}$$

Now we can use the fact that $(AB)^{-1} = B^{-1}A^{-1}$, as long as A and B are invertible, but we have to be careful! X^T and X are *not* invertible (they aren't square matrices), but $X^T X$ is square (and, presumably, invertible). C is also invertible. So, iteratively applying the fact, we get $(CX^T X C)^{-1} = C^{-1}(X^T X)^{-1}(C^T)^{-1}$, and we get:

$$= C^{-1}(X^T X)^{-1} (C^T)^{-1} C X^T \vec{y}$$

Since C is diagonal, $C^T = C$, so we can simplify:

$$\begin{aligned} &= C^{-1}(X^T X)^{-1} C^{-1} C X^T \vec{y} \\ &= C^{-1} \underbrace{(X^T X)^{-1} X^T \vec{y}}_{\vec{w}^*} \\ &= C^{-1} \vec{w}^* \end{aligned}$$

- c) Consider a new point $\vec{x}$ for which we want to make a prediction.

The prediction made by the original model is $\vec{w}^* \cdot \text{Aug}(\vec{x})$.

The prediction made by the new model trained on scaled data is $\vec{w}_C^* \cdot \text{Aug}(\vec{x}_C)$, where $\vec{x}_C$ is $\vec{x}$ scaled by the same factors as the training data. It can be shown that $\text{Aug}(\vec{x}_C) = C \text{Aug}(\vec{x})$.

Using these facts and the result of the previous parts, show that $\vec{w}^* \cdot \text{Aug}(\vec{x}) = \vec{w}_C^* \cdot \text{Aug}(\vec{x}_C)$; that is, both models make exactly the same prediction, and scaling had no effect.

Hint: The inverse of a diagonal matrix is also diagonal. The transpose of a diagonal matrix is the same as the original matrix.

Solution: Since $\vec{w}_C^* = C^{-1} \vec{w}^*$ and $\text{Aug}(\vec{x}_C) = C \text{Aug}(\vec{x})$, we have:

$$\vec{w}_C^* \cdot \text{Aug}(\vec{x}_C) = C^{-1} \vec{w}^* \cdot C \text{Aug}(\vec{x})$$

We want to somehow “cancel” the C and C^{-1} terms, which means getting them next to one another. For this, we can use the fact that $\vec{a} \cdot \vec{b} = \vec{a}^T \vec{b}$ for vectors $\vec{a}$ and $\vec{b}$ to write:

$$\begin{aligned} &= (C^{-1} \vec{w}^*)^T (C \text{Aug}(\vec{x})) \\ &= (\vec{w}^*)^T (C^{-1})^T C \text{Aug}(\vec{x}) \end{aligned}$$

Because C is diagonal, $(C^{-1})^T = (C^T)^{-1} = C^{-1}$, so we can simplify:

$$\begin{aligned} &= (\vec{w}^*)^T C^{-1} C \text{Aug}(\vec{x}) \\ &= (\vec{w}^*)^T \text{Aug}(\vec{x}) \\ &= \vec{w}^* \cdot \text{Aug}(\vec{x}) \end{aligned}$$

Problem 2.

Let $f(\vec{z}) = e^{z_1} + e^{z_2} + e^{z_3} + (z_1 - 1)^2 + \|\vec{z}\|^2$ be a function of $\vec{z} = (z_1, z_2, z_3)^T$.

Find a minimizer of this function using gradient descent (implemented with code; don't run GD by hand!). Report:

- The initial point, $\vec{z}^{(0)}$;
- your choice of step size;
- the stopping criterion you chose and number of iterations taken to reach the criterion;
- the minimizer you found;
- the minimum value of the function.

Include your code.

Solution: First, we do a little calculus to find the gradient of our function. The $\|\vec{z}\|^2$ term might throw you off, but remember that $\|\vec{z}\|^2 = z_1^2 + z_2^2 + z_3^2$. So we have:

$$f(\vec{z}) = e^{z_1} + e^{z_2} + e^{z_3} + (z_1 - 1)^2 + z_1^2 + z_2^2 + z_3^2.$$

With this, the partial derivatives are:

$$\begin{aligned}\frac{\partial f}{\partial z_1} &= e^{z_1} + 2(z_1 - 1) + 2z_1 \\ &= e^{z_1} + 4z_1 - 2\end{aligned}$$

$$\frac{\partial f}{\partial z_2} = e^{z_2} + 2z_2$$

$$\frac{\partial f}{\partial z_3} = e^{z_3} + 2z_3$$

Next, we code this up into a `gradient` function that computes the gradient.

```
def gradient(z):
    z_1, z_2, z_3 = z
    return np.array([
        np.exp(z_1) + 4*z_1 - 2,
        np.exp(z_2) + 2*z_2,
        np.exp(z_3) + 2*z_3
    ])
```

From here, we use the `gradient_descent` function from lecture with a starting location of $(0, 0, 0)$, and a step size of 0.1. As our stopping criterion, we'll stop when the norm of the gradient is less than 10^{-6} .

```
def gradient_descent(
    gradient, z_0, learning_rate, stop_threshold
):
    z = z_0
    i = 1
    while True:
        print(f"Iteration {i}: \t{z = }\t{f(z) = }")
        i += 1
        z_new = z - learning_rate * gradient(z)
```

```
    if np.linalg.norm(z_new - z) < stop_threshold:
        break
    z = z_new
return z_new
```

```
z_opt = gradient_descent(gradient, np.array([0, 0, 0]), 0.1, 1e-6)
```

With this, we find a minimizer of approximately $(0.196, -0.352, -0.352)^T$ in 39 iterations. At this point, the function value is approximately 3.556.

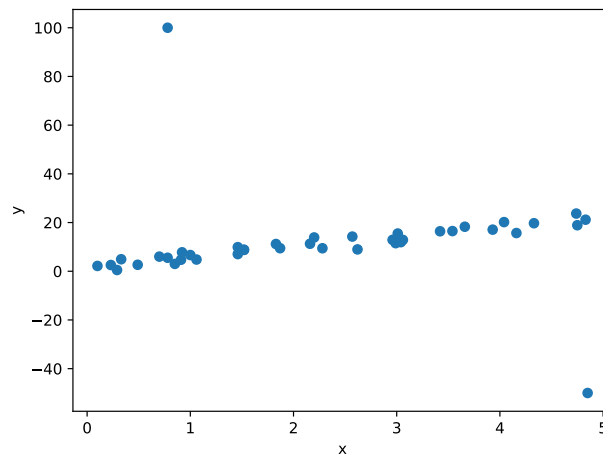
Problem 3.

The file at the link below contains a simple data set suitable for regression.

https://f000.backblazeb2.com/file/jeldridge-data/002-regression_outlier/data.csv

The first column contains the x values (the predictor variable) and the second column contains the y values (the target).

The plot below shows the data:



Notably, the data contains outliers which may affect our regression.

Fit a linear function of the form $w_0 + w_1x$ by minimizing the mean squared error. Report w_0 and w_1 , plot the fitted line, and include your code.

Do not use `sklearn`, `scipy`, or any library except for `numpy` and `matplotlib` for this problem. You may use the code given in lecture for least squares regression.

Solution:

```
# load the data
X, y = np.loadtxt('./data.csv', delimiter=',').T

# make an augmented design matrix by adding a column of ones
# to X
X_aug = np.column_stack((
    np.ones_like(X), X
))

# fit by minimizing squared error
w_sq = np.linalg.lstsq(X_aug, y)[0]
```

We find, approximately: $w_0 \approx 11.26$ and $w_1 \approx 0.18$.

Problem 4.

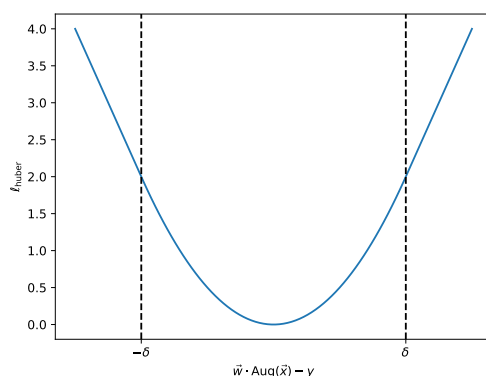
In the previous problem, you performed least squares regression on a data set.

As you can see, the data contains outliers which may affect our regression. In the previous homework, you should have found that least squares regression is not robust to these outliers.

The *Huber loss* is a loss function used in regression that is less sensitive to outliers than the square loss. It can be thought of as a “mix” between the square loss and the absolute loss. For linear prediction functions $H(\vec{x}) = \vec{w} \cdot \text{Aug}(\vec{x})$, the Huber loss is defined as:

$$\ell_{\text{huber}}(\text{Aug}(x) \cdot \vec{w}, y) = \begin{cases} \frac{1}{2}(\text{Aug}(x) \cdot \vec{w} - y)^2, & \text{if } |\text{Aug}(x) \cdot \vec{w} - y| \leq \delta, \\ \delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2, & \text{if } \text{Aug}(x) \cdot \vec{w} - y > \delta, \\ -\delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2, & \text{if } \text{Aug}(x) \cdot \vec{w} - y < -\delta, \end{cases}$$

A plot of the Huber loss is shown below.



Note that, despite being a piecewise function, the Huber loss is differentiable (unlike the absolute loss). However, there is no closed-form solution for the minimizer of the empirical risk with respect to the Huber loss, so if we want to train a regression model using this loss we need to use an iterative optimization algorithm, like gradient descent.

- a) Write a function to compute the risk of a linear prediction function $w_0 + w_1x$ with respect to the Huber loss (with $\delta = 1$) for the data given above. Use your function to compute the risk for $\vec{w} = (20, -1)$. Report the risk, and include your code.

Solution:

```
def huber_loss(w, x, y, delta=1):
    """Computes the Huber loss."""
    z = w @ x - y
    if np.abs(z) <= delta:
        return 0.5 * z**2
    elif z > delta:
        return delta * (z - 0.5 * delta)
    else: # np.abs(z) < -delta
        return -delta * (z + 0.5 * delta)

def huber_risk(w):
    """Computes the risk w.r.t. the Huber loss on the data set."""
```

```
return np.mean([huber_loss(w, X[i], y[i]) for i in range(len(X))])
```

Self-check: By running `huber_risk([20, -1])`, we find that the risk of the linear prediction function $20 - x$ with respect to the Huber loss is approximately 11.085.

- b) The gradient of the Huber loss with respect to $\vec{w}$ is also a piecewise function. Compute this gradient. Note that since the Huber loss is differentiable, the gradient can be computed by separately computing the gradient within each piece of the piecewise function. You may use any of the matrix-vector calculus rules we've seen in class to help you compute the gradient. For example, you may use the fact that $\frac{d}{d\vec{w}} \vec{w} \cdot \text{Aug}(\vec{x}) = \text{Aug}(\vec{x})$.

Solution: In the first case, where $|\text{Aug}(x) \cdot \vec{w} - y| \leq \delta$, the Huber loss is equal to $\frac{1}{2}(\text{Aug}(x) \cdot \vec{w} - y)^2$. Differentiating using the chain rule, we have:

$$\begin{aligned} \frac{d}{d\vec{w}} \frac{1}{2}(\text{Aug}(x) \cdot \vec{w} - y)^2 &= (\text{Aug}(x) \cdot \vec{w} - y) \frac{d}{d\vec{w}} (\text{Aug}(x) \cdot \vec{w} - y) \\ &= (\text{Aug}(x) \cdot \vec{w} - y) \text{Aug}(x) \end{aligned}$$

We recognize this as being the same as the gradient of the square loss (which it should be, since the Huber loss is equal to the square loss when $|\text{Aug}(x) \cdot \vec{w} - y| \leq \delta$).

For the second case, where $\text{Aug}(x) \cdot \vec{w} - y > \delta$, the Huber loss is $\delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2$. Differentiating this, we have:

$$\frac{d}{d\vec{w}} \delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2 = \delta \text{Aug}(x)$$

Finally, for the third case, where $\text{Aug}(x) \cdot \vec{w} - y < -\delta$, the Huber loss is $-\delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2$. Differentiating this, we have:

$$\frac{d}{d\vec{w}} -\delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2 = -\delta \text{Aug}(x)$$

So, putting it all together, the gradient of the Huber loss with respect to $\vec{w}$ is:

$$\frac{d}{d\vec{w}} \ell_{\text{huber}}(\text{Aug}(x) \cdot \vec{w}, y) = \begin{cases} (\text{Aug}(x) \cdot \vec{w} - y) \text{Aug}(x), & \text{if } |\text{Aug}(x) \cdot \vec{w} - y| \leq \delta, \\ \delta \text{Aug}(x), & \text{if } \text{Aug}(x) \cdot \vec{w} - y > \delta, \\ -\delta \text{Aug}(x), & \text{if } \text{Aug}(x) \cdot \vec{w} - y < -\delta, \end{cases}$$

- c) Write a function that computes the gradient of the empirical risk with respect to the Huber loss with $\delta = 1$ for a linear prediction function $w_0 + w_1x$. Use your function to compute the gradient for $\vec{w} = (20, -1)$. Show your code.

Solution:

```
def gradient_of_huber_loss(w, x, y, delta=1):
    """Computes the gradient of the Huber loss."""
    z = w @ x - y
    if np.abs(z) <= delta:
        return z * x
    elif z > delta:
        return delta * x
    else: # np.abs(z) < -delta
        return -delta * x
```

```
def gradient_of_empirical_risk(w):
    """Computes the gradient of the risk w.r.t. the Huber loss on the data set.

    Remember, the gradient of the empirical risk is the average of the gradients
    of the individual losses.

    """
    return np.mean(
        [gradient_of_huber_loss(w, X[i], y[i]) for i in range(len(X))], axis=0
```

Computing the gradient at $\vec{w} = (20, -1)$ is done by evaluating:

```
        gradient_of_empirical_risk([20, -1])
```

This gives us the gradient $\frac{dR}{d\vec{w}}(20, -1) = (0.532, 0.477)^T$.

- d) Implement and run gradient descent to minimize the empirical risk with respect to the Huber loss (using $\delta = 1$) on the data set above. In addition to your code, include:
- The optimal solution found by gradient descent;
 - A plot of the empirical risk of $\vec{w}^{(t)}$ at each iteration. In other words, include a plot whose horizontal axis is iteration number, t , and whose vertical axis measures the empirical risk of $\vec{w}^{(t)}$.
 - A plot of the data and the linear prediction function corresponding to the optimal solution.

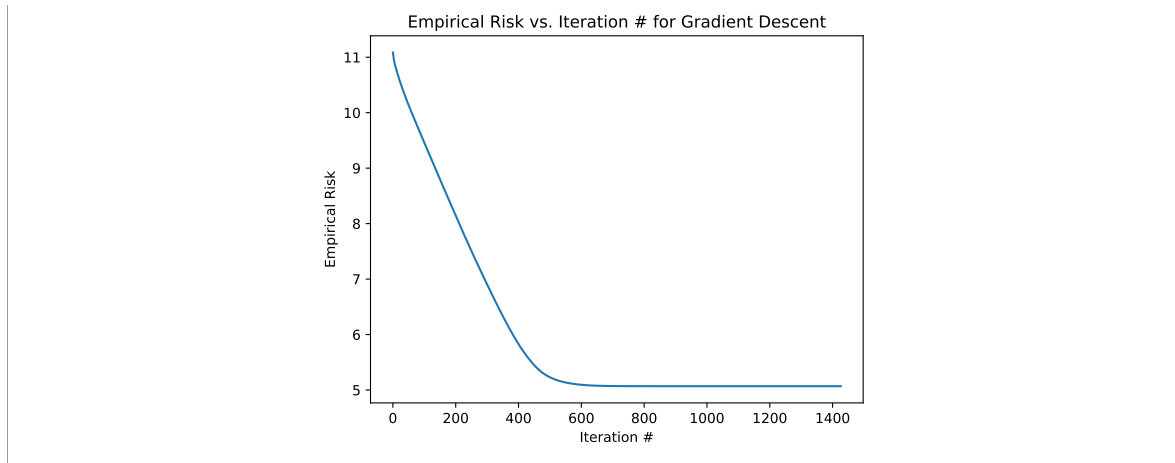
Solution:

```
def gradient_descent(gradient, w_0, learning_rate, stop_threshold):
    w = w_0
    i = 1
    risk_history = []
    while True:
        current_risk = huber_risk(w)
        risk_history.append(current_risk)
        print(f"Iteration {i}: \t{w = } \t{current_risk = }")
        i += 1
        w_new = w - learning_rate * gradient(w)
        if np.linalg.norm(w_new - w) < stop_threshold:
            break
        w = w_new
    return w_new, risk_history
```

We run gradient descent with:

```
w_gd, gd_risk_history = gradient_descent(
    gradient_of_empirical_risk, np.array([20, -1]), 0.1, 1e-6
)
```

We find a solution of $(2.287, 3.887)^T$ with an empirical risk of 5.068 in about 1,400 iterations. The plot of risk at each iteration is shown below:



- e) Implement and run *stochastic* gradient descent with a batch size of 8 to minimize the empirical risk with respect to the Huber loss (using $\delta = 1$) on the data set above.

Include:

- The optimal solution found by SGD;
- A plot of the empirical risk of $\vec{w}^{(t)}$ at each iteration. In other words, include a plot whose horizontal axis is iteration number, t , and whose vertical axis measures the empirical risk of $\vec{w}^{(t)}$.

Note that you should plot the risk with respect to the whole data set, and not the risk with respect to the random batch (the latter will be very noisy, and it's the former that we're trying to optimize).

Solution: There's a trick to implementing stochastic gradient descent: we just implement a `stochastic_gradient` function that computes the gradient for a random batch, and pass this function to `gradient_descent` in place of the usual `gradient` function.

Here is what that looks like:

```
def stochastic_gradient(w, batch_size=8):
    """Computes a stochastic gradient from `batch_size` random examples."""
    # select a random batch of data points
    batch = np.random.choice(len(X), batch_size, replace=True)
    X_batch = X[batch]
    y_batch = y[batch]

    return np.mean(
        [
            gradient_of_huber_loss(w, X_batch[i], y_batch[i])
            for i in range(len(X_batch))
        ],
        axis=0,
    )
```

We run stochastic gradient descent with:

```
w_sgd, sgd_risk_history = gradient_descent(
    stochastic_gradient, np.array([20, -1]), 0.001, 1e-6
)
```

We find a solution of $(2.319, 3.871)^T$ with an empirical risk of 5.069 in 60,000 iterations.

The plot of risk at each iteration is shown below:

